

Лабораторная работа №1. Настройка среды разработки и первый HTTP-сервер

Цель работы: Освоить установку инструментов для бэкенд-разработки, создать и запустить минимальный веб-сервер, обрабатывающий GET-запросы, понять концепцию эндпоинтов, научиться настраивать автоматический перезапуск сервера.

Краткое содержание лабораторной работы: Настройка среды разработки для бэкенд-приложений. Создание первого HTTP-сервера с использованием Express (Node.js) или Flask (Python). Обработка GET-запросов, отправка ответов в формате текста и JSON. Настройка автоматического перезапуска сервера при изменениях кода.

Теоретическое обоснование

1. Что такое бэкенд-разработка

Бэкенд (backend) - это серверная часть веб-приложения, которая отвечает за обработку запросов от клиентов, взаимодействие с базами данных, выполнение бизнес-логики и формирование ответов. Бэкенд работает на сервере и не виден пользователю напрямую.

Клиент-серверная архитектура

В веб-разработке взаимодействие строится по модели «клиент-сервер»:

- **Клиент** (браузер, мобильное приложение) отправляет HTTP-запрос.
- **Сервер** (бэкенд-приложение) принимает запрос, обрабатывает его и отправляет HTTP-ответ.

2. Эндпоинты и маршруты

Эндпоинт (endpoint) - это конкретный адрес (URL) вместе с методом HTTP, по которому клиент обращается к серверу для получения ресурса. Например: `GET /api/users`.

Маршрут (route) - это механизм сопоставления пути запроса с кодом обработчика на стороне сервера.

Протокол HTTP и метод GET

HTTP - основной протокол передачи данных в интернете. В данной работе мы сосредоточимся на методе **GET**. Он используется исключительно для **запроса/получения** данных от сервера и не должен изменять состояние сервера (свойство идемпотентности).

3. Что такое JSON

JSON (JavaScript Object Notation) - текстовый формат обмена данными, который стал стандартом де-факто для REST API. Пример ответа:

```
{
  "status": "ok",
  "message": "Сервер работает"
}
```

4. Выбор технологического стека

В этой лабораторной работе вы можете выбрать один из двух языков:

- Node.js + Express: самый популярный инструмент на JS, упрощает переход между фронтендом и бэкендом.

Node.js - это программная платформа, которая позволяет запускать код на **JavaScript** не только в браузере, но и на сервере.

Express - это минималистичный веб-фреймворк для **Node.js**, который помогает создавать серверные приложения (бэкенд) и API. Он упрощает обработку HTTP-запросов, маршрутизацию, работу с middleware и формирование ответов.

Простыми словами: Express - это инструмент, который превращает Node.js в полноценный веб-сервер, способный обрабатывать запросы от браузеров и мобильных приложений.

- Python + Flask: легкий фреймворк на самом популярном языке для бэкенда.

5. Автоматический перезапуск (Hot Reload)

При разработке мы часто меняем код. Чтобы не останавливать и не запускать сервер вручную каждый раз, мы используем:

- Для Node.js: `nodemon`
- Для Python: режим `debug=True` во Flask.

Пример выполнения

Вариант 1. Node.js + Express

Для начала необходимо установить Node.js

Перейдите на сайт: <https://nodejs.org/>

Скачайте LTS-версию (Long Term Support - стабильная и поддерживаемая)

Запустите установщик и следуйте инструкциям

Важно: При установке на Windows обязательно отметьте галочку "Add to PATH"

Шаг 1. Создание проекта

При создании проекта используем терминал VS Code

```
mkdir lab1-backend
cd lab1-backend
npm init -y
npm install express
npm install --save-dev nodemon
```

- `npm init -y` - Инициализация `package.json` (соглашаемся со значениями по умолчанию)
- `npm install express` - Установка Express
- `npm install --save-dev nodemon` - Установка `nodemon` для автоматического перезапуска (в dev-зависимости)

В VS Code и установите следующие расширения (нажмите `Ctrl+Shift+X` или иконку расширений слева):

Для Node.js + Express:

- **ES7** - полезные сниппеты
- **React/Redux/React-Native snippets** - полезные сниппеты (не обязательно, но пригодятся в будущем)
- **Prettier** - форматирование кода
- **JavaScript (ES6) code snippets**
- **npm Intellisense** - автодополнение для npm
- **Thunder Client** - альтернатива Postman для тестирования API
- **REST Client** - тестирование API прямо из VS Code

Шаг 2. Создание сервера (app.js)

```
const express = require('express');
const app = express();
const port = 3000;

// Консольное логирование каждого запроса (для продвинутого уровня)
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// 1. Текстовый эндпоинт
app.get('/', (req, res) => {
  res.send('Добро пожаловать на базовый сервер!');
});

// 2. JSON эндпоинт 1
```

```
app.get('/api/status', (req, res) => {
  res.json({ status: 'ok', uptime: process.uptime() });
});

// 3. JSON эндпоинт 2 (для среднего уровня)
app.get('/api/info', (req, res) => {
  res.json({ author: 'Student', version: '1.0.0' });
});

// 4. JSON эндпоинт с параметром (для продвинутого уровня)
app.get('/api/users/:id', (req, res) => {
  res.json({
    message: 'Информация о пользователе',
    userId: req.params.id
  });
});

// Обработка 404 (для среднего и продвинутого уровня)
app.use((req, res) => {
  res.status(404).json({ error: 'Маршрут не найден' });
});

app.listen(port, () => {
  console.log(`Сервер запущен на http://localhost:${port}`);
});
```

Шаг 3. Настройка package.json и запуск

Откройте package.json. Он будет примерно таким:

```
{ } package.json > ...
1  {
2    "name": "lab1-backend",
3    "version": "1.0.0",
4    "description": "",
5    "main": "app.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "dev": "nodemon app.js",
9      "start": "node app.js"
10   },
11   "author": "",
12   "license": "ISC",
13   "dependencies": {
14     "express": "^4.21.2"
15   },
16   "devDependencies": {
17     "nodemon": "^3.1.14"
18   }
19 }
20
```

Добавьте скрипт `"dev": "nodemon app.js"` и запустите: `npm run dev`.

Выполнение команды `npm run dev` будет выглядеть как на рисунке ниже.

```
D:\lab1-backend>npm run dev

> lab1-backend@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Сервер запущен на http://localhost:3000
[2026-09-02T20:21:17.252Z] GET /
[2026-09-02T20:21:17.329Z] GET /favicon.ico
█
```

Шаг 4. Проверка

Последовательно проверьте в браузере все эндпоинты.

В браузерной адресной строке введите:

1. <http://localhost:3000>
2. <http://localhost:3000/api/status>
3. <http://localhost:3000/api/info>
4. <http://localhost:3000/api/users/5>
5. <http://localhost:3000/api/users/123>
6. <http://localhost:3000/api/users/ivan>
7. <http://localhost:3000/anything-else>

При проверке эндпоинтов в терминале появляется лог

```
[node@lon] starting node app.js
Сервер запущен на http://localhost:3000
[2026-09-02T20:56:00.031Z] GET /api/status
```

Итоговая структура проекта:

```
D:\lab1-backend\
├─ node_modules\      # Папка с библиотеками (создана автоматически)
├─ app.js              # Код сервера
├─ package.json        # Описание проекта
└─ package-lock.json   # Фиксация версий (создан автоматически)
```

Вариант 2. Python + Flask

Шаг 1. Создание проекта

```
mkdir lab1-backend
cd lab1-backend
python -m venv venv
# Активировать venv
pip install flask
```

Шаг 2. Создание сервера (app.py)

```
from flask import Flask, jsonify, request
import time

app = Flask(__name__)

# Консольное логирование запроса(для продвинутого уровня)
@app.before_request
def log_request():
```

```

print(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] {request.method}
{request.path}")

# 1. Текстовый эндпоинт
@app.route('/')
def home():
    return 'Добро пожаловать на базовый сервер!'

# 2. JSON эндпоинт 1
@app.route('/api/status')
def status():
    return jsonify({ "status": "ok", "framework": "Flask" })

# 3. JSON эндпоинт 2(для среднего уровня)
@app.route('/api/info')
def info():
    return jsonify({ "author": "Student", "version": "1.0.0" })

# 4. JSON эндпоинт с параметром(для продвинутого уровня)
@app.route('/api/users/<int:user_id>')
def get_user(user_id):
    return jsonify({ "message": "Информация о пользователе", "userId": user_id })

# Обработка 404
@app.errorhandler(404)
def not_found(error):
    return jsonify({ "error": "Маршрут не найден" }), 404

if __name__ == '__main__':
    app.run(port = 3000, debug = True)

```

Запуск: `python app.py`

Индивидуальные задания

Данные в ответах JSON могут быть статичными (захардкоженными строками или объектами).

Задание 1. Базовый уровень

Требования: 2 эндпоинта (1 текстовый на корневом маршруте / и 1 JSON). Настроить автоматический перезапуск.

Вариант	Корневой ответ	JSON-эндпоинт	Пример статических данных JSON
---------	----------------	---------------	--------------------------------

1	"Добро пожаловать!"	/api/time	{"time": "12:00", "timezone": "UTC"}
2	"Hello, World!"	/api/date	{"date": "2024-01-01", "day": "Monday"}
3	"Привет, мир!"	/api/info	{"server": "Node", "port": 3000}
4	"Welcome!"	/api/health	{"status": "healthy", "uptime": "100s"}
5	"Сервер работает"	/api/version	{"version": "1.0.0", "release": "stable"}
6	"API Gateway"	/api/status	{"status": "online", "connections": 5}
7	"Мой бэкенд"	/api/name	{"app_name": "Lab1", "author": "Student"}
8	"Hello from backend"	/api/env	{"environment": "development"}
9	"Привет из бэкенда"	/api/system	{"os": "Linux", "arch": "x64"}
10	"Backend is ready"	/api/ready	{"ready": true, "message": "All good"}
11	"Сервер запущен"	/api/ping	{"message": "pong", "delay": "10ms"}
12	"Welcome to my API"	/api/docs	{"docs_url": "/api/docs", "version": "v1"}
13	"Документация API"	/api/features	{"features": ["auth", "data"]}
14	"Hello, студент!"	/api/group	{"group": "ИБТ-31", "year": 2024}
15	"Мой первый сервер"	/api/author	{"name": "Иван", "surname": "Иванов"}

Задание 2. Средний уровень

Требования: 3 эндпоинта (1 текстовый на / и 2 различных JSON-эндпоинта). Настроить автоматический перезапуск. Добавить кастомную обработку ошибки 404 (Not Found) с возвратом JSON формата {"error": "Not Found"}.

Вариант	JSON-эндпоинт 1 (сущность)	JSON-эндпоинт 2 (справочник)
1	/api/users (стат. список)	/api/roles (список ролей)
2	/api/posts (стат. список)	/api/tags (список тегов)
3	/api/books (стат. список)	/api/genres (список жанров)
4	/api/cities (стат. список)	/api/countries (список стран)
5	/api/movies (стат. список)	/api/directors (список режиссеров)
6	/api/students (стат. список)	/api/groups (список групп)
7	/api/orders (стат. список)	/api/statuses (статусы заказов)
8	/api/products (стат. список)	/api/categories (список категорий)
9	/api/employees (стат. список)	/api/departments (список отделов)
10	/api/events (стат. список)	/api/locations (список локаций)
11	/api/songs (стат. список)	/api/albums (список альбомов)
12	/api/recipes (стат. список)	/api/ingredients (список продуктов)
13	/api/games (стат. список)	/api/platforms (список платформ)
14	/api/hotels (стат. список)	/api/stars (варианты звездности)
15	/api/flights (стат. список)	/api/airlines (список авиакомпаний)

Задание 3. Повышенный уровень

Требования:

- 5 эндпоинтов (1 текстовый, 2 простых JSON-эндпоинта, 1 JSON-эндпоинт с параметром в пути, 1 обработчик 404).
- Автоматический перезапуск.
- Реализация простого консольного логирования всех входящих запросов (метод + URL + время).

Вариант	Сущность 1 (коллекция)	Сущность 2 (коллекция)	Эндпоинт с параметром (ID)
1	/api/tasks	/api/projects	/api/tasks/:id
2	/api/users	/api/companies	/api/users/:id
3	/api/products	/api/brands	/api/products/:id
4	/api/books	/api/authors	/api/books/:id
5	/api/movies	/api/cinemas	/api/movies/:id
6	/api/orders	/api/clients	/api/orders/:id
7	/api/students	/api/courses	/api/students/:id
8	/api/posts	/api/comments	/api/posts/:id
9	/api/tickets	/api/events	/api/tickets/:id
10	/api/employees	/api/offices	/api/employees/:id
11	/api/recipes	/api/chefs	/api/recipes/:id
12	/api/matches	/api/teams	/api/matches/:id
13	/api/rooms	/api/hotels	/api/rooms/:id
14	/api/dishes	/api/menus	/api/dishes/:id
15	/api/cars	/api/dealers	/api/cars/:id

(Для параметризованного эндпоинта достаточно возвращать JSON с подтверждением, например: {"requestedId": 5, "status": "success"})

Контрольные вопросы

1. Что такое клиент-серверная архитектура?
2. Какой протокол используется для общения клиента и сервера в вебе?
3. Что такое эндпоинт (endpoint)?

4. Какую структуру имеет HTTP-запрос и HTTP-ответ? Что такое код состояния (status code)?
5. Что такое JSON? Почему он популярен в веб-разработке?
6. Как запустить сервер на Node.js или Flask?
7. Что такое автоматический перезапуск сервера и зачем он нужен? Какой пакет используется для Node.js?
8. Какие коды состояния HTTP вы знаете? За что отвечает код 404?
9. Что такое middleware в Express / декораторы запросов во Flask (для продвинутого уровня)?
10. Как получить параметр из URL-пути (например, ID пользователя) в вашем фреймворке?

Критерии оценивания

Критерий	Базовый уровень (max 30)	Средний уровень (max 45)	Продвинутый уровень (max 60)
Настройка среды и авто-перезапуск	10	10	10
Возврат текстового ответа	10	5	5
Создание простых JSON-эндпоинтов	10	15 (за 2 шт.)	15 (за 2 шт.)
Обработка 404 ошибки	—	15	10
Эндпоинт с параметром в пути	—	—	10
Консольное логирование запросов	—	—	10
Итого	30	45	60

Шкала перевода баллов в оценку:

Оценка	Базовый (30)	Средний (45)	Продвинутый (60)
5 (отлично)	—	—	54-60
4 (хорошо)	—	40-45	46-53
3 (удовлетворительно)	24-30	34-39	38-45
2 (неудовлетворительно)	0-23	0-33	0-37

Требования к отчёту

Отчёт должен быть представлен в формате **Markdown** и содержать:

1. **Титульный лист** - название работы, ФИО, группа, вариант.
2. **Цель работы** - формулировка цели.
3. **Теоретическое обоснование** - краткое изложение ключевых понятий (2-3 абзаца).
4. **Выполнение практического примера** - код сервера, скриншоты результатов запросов (curl или браузер), команды для запуска.
5. **Выполнение индивидуального задания** - код сервера, скриншоты работы всех эндпоинтов.
6. **Ответы на контрольные вопросы** - не менее 5 вопросов по своему уровню.
7. **Вывод** - что было сделано, что нового узнали, с какими трудностями столкнулись.
8. **Список использованных источников** - ссылки на документацию.

Структура репозитория для лабораторных работ семестра

```
student-backend-course/
├── README.md # Описание курса, навигация по работам
├── .gitignore # Игнорируемые файлы
├── lab1-first-http-server/
│   ├── README.md # Отчёт по ЛР №1
│   ├── app.js / app.py # Код сервера
│   ├── package.json / requirements.txt # Зависимости
│   ├── screenshots/ # Скриншоты работы
│   │   ├── root.png
│   │   ├── json.png
│   │   └── param.png
│   └── curl_commands.txt # Команды для тестирования
├── lab2-http-methods/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab3-routing-params/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab4-sql-basics/
│   ├── README.md
│   ├── init_db.sql
│   └── screenshots/
├── lab5-sql-from-server/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab6-crud-database/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab7-sql-filtering/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
└── lab8-sql-joins/
    ├── README.md
    └── app.js / app.py
```

```

├── screenshots/
├── lab9-rest-api-design/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab10-validation/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab11-password-hashing/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab12-session-auth/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab13-env-config/
│   ├── README.md
│   ├── app.js / app.py
│   ├── .env.example
│   └── screenshots/
├── lab14-logging/
│   ├── README.md
│   ├── app.js / app.py
│   └── screenshots/
├── lab15-postman-tests/
│   ├── README.md
│   └── screenshots/
├── lab16-frontend-integration/
│   ├── README.md
│   ├── app.js / app.py
│   ├── public/
│   │   └── index.html
│   └── screenshots/
├── lab17-deploy/
│   ├── README.md
│   └── screenshots/
├── lab18-final-project/
│   ├── README.md
│   ├── app.js / app.py
│   ├── public/
│   └── screenshots/
├── common/
│   ├── styles/ # Общие стили для оформления
│   └── templates/ # Шаблоны README
├── docs/
├── course_description.md # Описание курса
└── grading_rubric.md # Критерии оценивания

```

Описание ключевых файлов и папок

Файл/папка	Назначение
README.md	Главный файл курса с описанием, списком работ, навигацией
.gitignore	Исключает node_modules/, venv/, .env, *.log из репозитория
lab*/README.md	Отчёт по лабораторной работе (Markdown)
lab*/screenshots/	Скриншоты выполнения запросов, работы сервера
common/	Общие файлы для оформления всех работ
docs/	Документация курса, критерии оценивания

Пример оформления README.md для каждой работы

```
# Лабораторная работа №1: Настройка среды разработки и первый HTTP-сервер

**Студент:** Иванов Иван Иванович
**Группа:** ПИЖ-б-о-25-1
**Вариант:** 5
**Технология:** Node.js + Express

## Содержание
- [Цель работы] (#цель-работы)
- [Теоретическое обоснование] (#теоретическое-обоснование)
- [Выполнение практического примера] (#выполнение-практического-примера)
- [Выполнение индивидуального задания] (#выполнение-индивидуального-задания)
- [Контрольные вопросы] (#контрольные-вопросы)
- [Вывод] (#вывод)

## Код сервера
```javascript
const express = require('express');
// ...
```

## Скриншоты

### Корневой маршрут JSON-ответ

```

Рекомендуемые источники

1. **Node.js официальная документация** – https://nodejs.org/en/docs/
2. **Express официальная документация** – https://expressjs.com/
3. **Flask официальная документация** – https://flask.palletsprojects.com/
4. **HTTP протокол (MDN)** – https://developer.mozilla.org/ru/docs/Web/HTTP
5. **JSON (MDN)** – https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/JSON
6. **Nodemon документация** – https://nodemon.io/
7. **Postman документация** – https://learning.postman.com/docs/getting-started/introduction/
8. **REST API – основные принципы (GeeksforGeeks)** – https://www.geeksforgeeks.org/node-js/explain-the-concept-of-restful-apis-in-express
```